

Programmare un bot: laboratorio JavaScript e Node.js



Dal primo filtro al monitor asincrono • Percorso per studenti ITS

Questa guida insegna a costruire il programma. Presuppone variabili, condizioni e cicli in un qualsiasi linguaggio; spiega la sintassi JavaScript quando compare. I primi esercizi funzionano senza Vinted, Telegram o credenziali. I servizi reali arrivano dopo, quando sai spiegare e verificare la logica.

Il progetto osserva articoli, applica regole e notifica quelli nuovi. Non determina automaticamente il valore commerciale e non acquista. Vinted è il caso di studio per problemi comuni: HTTP, dati esterni, stato, errori e test.

Come studiare: esegui una fase, prevedi l'effetto di una modifica, poi verifica il risultato. Completa gli esercizi prima di leggere le soluzioni. Il codice eseguibile è in docs/laboratorio; bot.js è il programma reale. I frammenti indicati come estratti non sono programmi autonomi.

Percorso del laboratorio

1. Progettare prima di programmare
2. Strumenti, moduli e primo programma
3. Oggetti, array e funzioni pure
4. JSON e validazione al confine
5. HTTP: collegare client e server
6. Promise, async e await
7. Stato, Set e closure
8. Telegram come adapter di uscita
9. Cookie e adapter Vinted
10. Errori, retry e backoff
11. Scheduler e ciclo di vita
12. Test e dependency injection
13. Leggere il codice del bot reale
14. Soluzioni ragionate
15. Mini-progetto finale e metodo di lavoro

In appendice: cinque file completi da eseguire e modificare. Gli esempi locali non richiedono account o segreti.

1. Progettare prima di programmare

Trasforma la richiesta in input, trasformazioni e output. Input: ricerca, prezzo massimo e articoli. Output: una notifica per articolo compatibile non ancora notificato. Stato: ID notificati e cookie. Eventi: avvio, scadenza del timer, risposta HTTP.

controlla:

```
se un controllo è attivo: termina
ottieni gli articoli
per ogni articolo:
  se prezzo non valido: salta
  se prezzo oltre soglia: salta
  se ID già notificato: salta
  invia la notifica
registra ID dopo conferma
```

«Registra dopo conferma» è un requisito: registrare prima significherebbe perdere la notifica quando l'invio fallisce. Il pseudocodice rende visibile questa scelta prima di selezionare librerie.

Dividi le responsabilità: configurazione, acquisizione dati, filtro, formattazione, invio e coordinamento. Una funzione che conosce contemporaneamente cookie, prezzi, token e cron è difficile da provare. Separando il filtro dalla rete puoi verificarlo con un array scritto a mano.

Esercizio 1: scrivi input e output attesi per prezzo oltre soglia, ID duplicato e notifica fallita. Indica se lo stato deve cambiare.

2. Strumenti, moduli e primo programma

Node esegue JavaScript fuori dal browser; npm installa dipendenze. .mjs indica un modulo; in questo repository anche .js è un modulo perché package.json contiene "type": "module". import ed export collegano file e funzioni.

Dal Prompt dei comandi Windows, dentro il repository:

```
node --version
npm ci
node docs/laboratorio/01-filtro.mjs
```

Serve Node 22 o successivo. npm ci reinstalla le versioni del lockfile; non compila il tuo JavaScript. I primi laboratori usano solo moduli integrati.

Per un progetto nuovo usa una cartella separata, npm init -y e file .mjs. Aggiungi librerie quando servono: npm install dotenv node-cron tough-cookie. Non iniziare riscrivendo un parser di cookie o uno scheduler.

Il terminale esegue; l'editor modifica; gli strumenti sviluppatore del browser osservano le richieste di un sito. «Cannot find module .../non» significa che node ha ricevuto non come nome del file, non che la ricerca Vinted è fallita.

Esercizio 2: crea saluto.mjs con console.log('Ciao ITS') ed eseguilo. Rinomina il file e prova il vecchio comando. Distingui un percorso errato da un errore della logica.

3. Oggetti, array e funzioni pure

Un articolo è un oggetto con proprietà; una lista è un array. Una funzione può prendere la lista e restituirne un sottoinsieme senza modificarla.

```
const articoli = [
  { id: 1, titolo: 'Bracciale', prezzo: 20 },
  { id: 2, titolo: 'Collana', prezzo: 45 },
  { id: 3, titolo: 'Anello', prezzo: 25 }
];

function trovaOccasioni(lista, massimo) {
  return lista.filter(articolo => {
    return Number.isFinite(articolo.prezzo)
      && articolo.prezzo >= 0
      && articolo.prezzo <= massimo;
  });
}

console.log(trovaOccasioni(articoli, 25));
```

Risultato: oggetti con ID 1 e 3. filter visita ogni elemento e lo conserva se la funzione restituisce true. articolo => ... è una funzione passata come argomento, chiamata callback. Puoi ottenere lo stesso risultato con un ciclo e push.

La funzione è pura: dipende dagli argomenti, non usa rete o stato esterno e non modifica lista. Questo facilita i test. const impedisce di riassegnare una variabile, ma non congela l'oggetto: articoli.push(...) è ancora possibile. Usa let se devi riassegnare.

&& è AND logico con valutazione da sinistra a destra: se una condizione è falsa, le successive non vengono valutate. return restituisce il risultato e termina la funzione. filter crea un nuovo array, ma gli oggetti al suo interno restano gli stessi riferimenti: modificare un oggetto selezionato modifica anche quello presente nella lista originale.

Il file 01-filtro.mjs contiene una versione completa con un'asserzione. Esercizio 3: aggiungi prezzi 0, -1, NaN e 25.01. Prevedi il risultato e verifica. Riscrivi il filtro con for...of senza modificare l'array originale.

4. JSON e validazione al confine

JSON è testo strutturato. `JSON.parse` converte testo in dati; `JSON.stringify` converte dati in testo. Un oggetto JavaScript può contenere anche metodi e valori che JSON non rappresenta direttamente.

L'API può fornire:

```
const item = {
  id: 42,
  title: 'Bracciale',
  price: { amount: '20.00', currency_code: 'EUR' }
};
const amount = Number(item.price.amount);
console.log(amount <= 25); // true
```

Una conversione non è una validazione. `Number('')` e `Number(null)` valgono 0, mentre `Number(undefined)` è NaN. Un prezzo mancante non deve diventare un articolo gratis.

Estratto del bot:

```
const raw = typeof item.price === 'object'
  ? item.price?.amount : item.price;
const amount = raw === null || raw === undefined || raw === ''
  ? NaN : Number(raw);
if (!Number.isFinite(amount) || amount < 0) continue;
```

?. è optional chaining: se `price` è null o undefined non lancia un errore. ?? sceglie un valore alternativo per null e undefined; || lo sceglie per tutti i valori falsy, compresi 0 e stringa vuota. Non sono intercambiabili.

Il bot copre il formato osservato, ma non è un validatore universale: `Number('')` vale 0 e tipi inattesi richiedono più controlli. I test sui confini aiutano a trovare questi casi.

Esercizio 4: scrivi `leggiPrezzo(item)`, che restituisca un numero o null. Rifiuta stringhe vuote o di soli spazi, booleani, negativi e valori non numerici. Accetta '20.50' e 0. Lavora nel laboratorio senza cambiare subito `bot.js`.

5. HTTP: collegare client e server

Una richiesta contiene metodo, URL, intestazioni e talvolta corpo. GET richiede dati; POST li invia. La risposta contiene status, intestazioni e corpo. 200 indica successo HTTP, ma il contenuto deve essere controllato.

I parametri dopo ? formano la query string. URLSearchParams codifica spazi e simboli:

```
const url = new URL('http://127.0.0.1:3000/items');
url.search = new URLSearchParams({
  search_text: 'bracciale argento',
  price_to: '25'
}).toString();
console.log(url.href);
```

Il laboratorio usa HTTP solo su loopback, senza credenziali. Il servizio vero usa HTTPS. Esegui `node docs/laboratorio/02-http.mjs`: avvia un server su una porta libera, fa una richiesta, filtra i risultati e chiude il server. Il risultato contiene solo il bracciale da 20. Una seconda richiesta dimostra la gestione di 404.

`createServer` è il server, `fetch` il client. Anche nello stesso processo, comunicano con una vera richiesta HTTP. La porta non è fissa: `listen(0)` chiede al sistema di sceglierne una libera. Leggi come il server crea JSON e il client lo decodifica.

Esercizio 5: fai leggere al server `price_to` e filtrare prima di rispondere. Mantieni anche il filtro locale: il client deve restare corretto quando un servizio ignora un parametro.

6. Promise, async e await

La rete non produce un risultato immediato. Una Promise rappresenta un risultato futuro: pending, fulfilled oppure rejected. `async` fa restituire una Promise alla funzione. `await` attende la risoluzione e sospende la continuazione di quella funzione asincrona.

```
async function caricaArticoli(url) {
  const response = await fetch(url, {
    signal: AbortSignal.timeout(20_000)
  });
  if (!response.ok) throw new Error(`HTTP ${response.status}`);
  const data = await response.json();
  if (!Array.isArray(data.items)) {
    throw new Error('items deve essere un array');
  }
  return data.items;
}
```

Due attese: intestazioni della risposta, poi lettura e decodifica del corpo. `fetch` normalmente non rifiuta per 404 o 500: restituisce una `Response` con `ok=false`. Rifiuta per errori di connessione o `abort`. Controlla perciò `response.ok`.

`await` non blocca tutto Node: il motore può elaborare altri eventi durante l'attesa della rete. Un ciclo sincrono pesante, invece, occupa il thread JavaScript e ritarda timer e callback.

Un `throw` dentro `async` produce una Promise rifiutata. Il chiamante la gestisce con `try/catch` attorno ad `await`. Senza `await` potresti stampare una Promise al posto degli articoli o non intercettare il rifiuto dove pensi.

Esercizio 6: aggiungi un ritardo di 500 ms al server prima della risposta. Stampa «prima», risultato, «dopo». Confronta con e senza `await`. Non usare un timer per supporre che la richiesta abbia finito: attendi la Promise.

7. Stato, Set e closure

Il filtro non ricorda il passato. Per evitare duplicati serve stato conservato tra controlli. Set memorizza valori unici; has consulta, add inserisce, size conta.

Una funzione può restituire un'altra funzione che mantiene accesso alle variabili esterne: è una closure. Estratto della factory del laboratorio:

```
function creaMonitor({ carica, invia, massimo }) {
  const notificati = new Set();
  let attivo = false;
  async function controlla() {
    if (attivo) return { saltato: true };
    attivo = true;
    try {
      const articoli = await carica();
      for (const articolo of trovaOccasioni(articoli, massimo)) {
        const id = String(articolo.id);
        if (notificati.has(id)) continue;
        await invia(articolo);
        notificati.add(id);
      }
    } finally {
      attivo = false;
    }
  }
  return { controlla, notificati };
}
```

La versione completa 03-monitor.mjs valida dipendenze, soglia e ID e restituisce un riepilogo. Esegui `node docs/laboratorio/03-demo.mjs`: primo controllo, una notifica simulata; secondo controllo, zero nuove notifiche.

`{ carica, invia, massimo }` nel parametro è destructuring: estrae tre proprietà dall'oggetto passato. `return { controlla, notificati }` è una forma abbreviata di `{ controlla: controlla, notificati: notificati }`. Nel laboratorio esponiamo il Set per osservarlo nei test; un'API più incapsulata potrebbe offrire solo conteggi o copie, evitando modifiche esterne accidentali.

finally viene eseguito anche dopo un errore. Senza finalmente liberare attivo, un invio fallito potrebbe bloccare tutti i controlli futuri. Il blocco vale nel processo corrente, non coordina due copie indipendenti.

String rende 42 e '42' la stessa chiave. Il Set si azzerà al riavvio e cresce nel tempo. La factory lascia propagare gli errori: il chiamante dovrà decidere quando riprovare.

Esercizio 7: simula un invio fallito al primo tentativo e riuscito al secondo. Dopo il fallimento l'ID non deve essere nel Set e il blocco deve essere liberato. Sposta poi add prima di invia e osserva quale requisito rompi.

8. Telegram come adapter di uscita

Nel laboratorio invia stampa testo. Sostituirla con una funzione che esegue POST realizza un adapter: stesso contratto verso il monitor, implementazione diversa.

Estratto da usare solo con una configurazione reale:

```
async function inviaTelegram(token, chatId, testo) {
  const endpoint = `https://api.telegram.org/bot${token}/sendMessage`;
  const response = await fetch(endpoint, {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify({ chat_id: chatId, text: testo }),
    signal: AbortSignal.timeout(20_000)
  });
  if (!response.ok) throw new Error(`Telegram HTTP ${response.status}`);
  const data = await response.json();
  if (data.ok !== true) throw new Error('Invio non confermato');
}
```

JSON.stringify produce il corpo; Content-Type dice al server come leggerlo. chat_id indica il destinatario; token autorizza il bot. Non stampare l'endpoint: contiene il token.

Le stringhe tra backtick sono template literal: \${espressione} inserisce un valore. Non confonderle con comandi shell. L'operatore ... nella gestione delle intestazioni è spread: copia proprietà da un oggetto dentro un altro. cookie ? oggetto : {} è un ternario, cioè una scelta tra due valori.

dotenv legge .env all'avvio. I segreti non sono costanti da pubblicare. .gitignore esclude .env, ma non impedisce di copiarne valori in log e screenshot. Un token esposto va revocato. .env.example contiene solo valori pubblici e segnaposto.

La formattazione dovrebbe essere una funzione pura separata dalla rete. Il bot reale usa testo semplice e limita le lunghezze dei campi. I dati degli annunci non devono diventare comandi shell, codice o HTML interpretato.

Esercizio 8: crea formattaMessaggio(articolo), verifica il testo con assert e passalo a un invio simulato. Normalizza prezzo e titolo prima della formattazione.

9. Cookie e adapter Vinted

Sostituisci la carica simulata con una sorgente reale solo dopo avere verificato il monitor. L'adapter deve ottenere sessione, costruire URL, verificare HTTP e JSON, poi restituire dati adatti alla logica interna.

Un cookie va conservato e rimandato rispettando dominio, percorso, scadenza e sicurezza. Node fetch non dispone automaticamente dell'archivio cookie di un browser. tough-cookie gestisce quelle regole.

Estratto di gestione cookie:

```
const cookie = await jar.getCookieString(url);
const response = await fetch(url, {
  headers: { ...(cookie ? { Cookie: cookie } : {}) }
});
for (const value of response.headers.getSetCookie()) {
  await jar.setCookie(value, response.url || url);
}
```

Il bot completo aggiunge User-Agent, Accept, lingua, Referer e timeout. refreshSession svuota il jar e visita la homepage. Un cookie del solo www non deve essere forzato sul dominio api. User-Agent non autentica e non garantisce accesso.

Il vecchio /api/v2/catalog/items restituiva 404. Gli strumenti del browser hanno mostrato <https://api.vinted.it/svc-catalogue/items> con 200. La diagnostica sul tuo PC ha poi verificato 20 articoli con price.amount e price.currency_code. È un endpoint osservato, non una API pubblica stabile.

La risposta non esponeva i vecchi size_title e brand_title: il codice mostra N/D quando mancano. Non inventare nuovi percorsi dei campi senza osservarne la struttura. Un 200 nel browser non garantisce accesso da Node con una sessione differente.

Esercizio 9: separa tre confini: risposta HTTP, oggetto dell'API, modello interno. Elenca i controlli di ciascuno e dove convertire un prezzo stringa in numero.

10. Errori, retry e backoff

401 indica un problema di sessione o autenticazione; 403 accesso negato; 404 risorsa non disponibile; 429 limitazione delle richieste. Un timeout non dice necessariamente se il server ha completato l'operazione.

Il bot rinnova i cookie dopo 401 e ritenta una sola volta, senza ricorsioni indefinite. Per 429 considera Retry-After, espresso in secondi o come data HTTP. CAPTCHA e blocchi non vanno aggirati disattivando verifiche.

Backoff significa attesa crescente. Estratto del bot:

```
failures += 1;
const delay = Math.max(
  error.retryAfter || 0,
  Math.min(30 * 60_000, 30_000 * 2 ** Math.min(failures, 6))
);
nextCheckAt = Date.now() + delay;
```

La base produce 60, 120, 240, 480, 960 e 1800 secondi, poi resta a 1800; Retry-After più lungo prevale. Un successo azzerà failures. Date.now e i timer lavorano in millisecondi: confondere 60 con 60_000 cambia l'attesa di un fattore mille.

Se Telegram accetta l'invio e la risposta si perde, il client può ritentare e duplicare il messaggio. Il Set non garantisce «exactly once». La consegna distribuita ha ambiguità che non si risolvono solo con una struttura dati locale.

Esercizio 10: estrai calcolaAttesa(numeroErrori, retryAfter) come funzione pura. Prova primo errore, sesto errore e Retry-After di un'ora. Confronta numeri senza aspettare davvero.

11. Scheduler e ciclo di vita

Prima verifica un singolo controllo, poi ripetilo. node-cron usa sei campi: secondi, minuti, ore, giorno del mese, mese, giorno della settimana.

```
import cron from 'node-cron';
const task = cron.schedule('*/*30 * * * *', () => {
  void monitor.check();
});
void monitor.check(); // controllo immediato
```

Nel bot check gestisce gli errori internamente. Nel laboratorio controlla li propaga: per collegarlo a cron serve catch, per esempio monitor.controlla().catch(...). void non intercetta errori: ignora soltanto il valore restituito.

Due callback possono partire mentre la prima è ancora in attesa. running o attivo impedisce sovrapposizioni. Se un controllo impiega 15 secondi e cron scatta ogni 10, alcune esecuzioni vengono saltate. Non equivale a completare una ricerca ogni 10 secondi.

Ctrl+C interrompe il processo. Il bot ferma cron e termina senza garantire il completamento degli invii in corso. Un servizio più robusto potrebbe aspettarli. Sospensione e ibernazione fermano lavoro e rete; il bot non recupera automaticamente tutti i controlli persi.

Esercizio 11: confronta intervallo fisso e ciclo che attende lavoro e poi pausa. Il primo programma istanti di partenza; il secondo include la durata del lavoro nel periodo. Scegli in base al requisito.

12. Test e dependency injection

Un test prepara, agisce e verifica. Deve fallire quando rompi un requisito. Non basta controllare che una funzione esista.

Il monitor riceve carica e invia: è dependency injection. I test forniscono funzioni simulate, senza Vinted o Telegram. Estratto:

```
const inviati = [];  
const monitor = creaMonitor({  
  massimo: 25,  
  carica: async () => [{ id: 1, titolo: 'A', prezzo: 20 }],  
  invia: async articolo => { inviati.push(articolo.id); }  
});  
await monitor.controlla();  
await monitor.controlla();  
assert.deepEqual(inviati, [1]);
```

Per provare sovrapposizioni usiamo una Promise controllabile. La prima carica aspetta finché il test non la sblocca. Nel frattempo una seconda chiamata deve essere saltata. Non servono sleep arbitrari.

Esegui `node --test docs/laboratorio/verifica.mjs`. Cinque test coprono filtro e soglia, duplicati, invio fallito, rilascio del blocco dopo errore di acquisizione e sovrapposizioni. `npm test` esegue i sei test del bot reale con HTTP simulato.

Livelli: test unitario per logica; server locale per HTTP; `node bot.js --check` per accesso e struttura reali; `npm start` per flusso completo e Telegram. Un livello non sostituisce gli altri.

Esercizio 12: cambia temporaneamente `<=` in `<` nel filtro. Il caso con prezzo alla soglia deve fallire. Se resta verde, il test non verifica quel requisito. Ripristina la condizione.

13. Leggere il codice del bot reale

Segui bot.js per responsabilità: readConfig converte e valida; createMonitor crea jar, Set e stato; vintedRequest gestisce HTTP e cookie; refreshSession rinnova; getItems cerca e controlla il JSON; notify invia; check coordina; inspect fornisce diagnostica senza Telegram.

Il blocco finale esegue il programma solo se bot.js è avviato direttamente. Confronta import.meta.url e pathToFileURL(process.argv[1]).href. I test possono importare funzioni senza avviare cron.

createMonitor(config, fetchFn = fetch) permette di sostituire fetch nei test. È lo stesso principio del laboratorio applicato al confine HTTP. La closure conserva stato; config raccoglie i valori comuni.

Esercizio 13: segui un articolo con carta e penna: arriva da items, ha prezzo valido, passa la soglia, non è nel Set, viene notificato e registrato. Ripeti con Telegram 429: quali righe salti e quali variabili cambi?

Limiti effettivi: una ricerca, una pagina, ID in memoria, stima di rivendita fissa opzionale, nessun acquisto automatico. Più ricerche, persistenza e adattamento marca/taglia sono modifiche future da progettare, non funzioni già presenti.

14. Soluzioni ragionate

1. Prezzo oltre soglia e duplicato: nessun invio, stato invariato. Invio fallito: ID non aggiunto, blocco liberato. Gli articoli già inviati prima dell'errore restano registrati.

2. `console.log('Ciao ITS')` stampa una riga. Dopo la rinomina il vecchio comando non trova il modulo: il codice non è stato eseguito. Correggi il percorso.

3. Zero e 25 passano; -1, NaN e 25.01 no. Un ciclo equivalente usa un array risultato e push quando la stessa condizione vale true.

4. Una soluzione più rigorosa:

```
function leggiPrezzo(item) {
  const raw = typeof item.price === 'object'
    ? item.price?.amount : item.price;
  if (typeof raw !== 'number' && typeof raw !== 'string') return null;
  if (typeof raw === 'string' && raw.trim() === '') return null;
  const value = Number(raw);
  return Number.isFinite(value) && value >= 0 ? value : null;
}
```

5. Leggi `url.searchParams.get('price_to')`, verifica presenza e validità, convertilo e filtra sul server. Il client mantiene la propria verifica.

6. Con `await`: prima, dati, dopo. Senza `await` il chiamante ottiene una Promise e procede senza aspettare. Usa `try/catch` con `await`, non un ritardo scelto a caso.

7. Nel fake invia un contatore lancia al primo tentativo. Dopo il primo fallimento Set vuoto; dopo il secondo tentativo ID presente. Spostare `add` prima dell'invio rompe il test. La verifica completa è allegata.

8. Usa una template string con titolo e `prezzo.toFixed(2)`. Verifica il testo esatto con un articolo noto. Normalizza prima: il formatter non deve inventare un prezzo mancante.

9. HTTP: status e timeout. API: array items e tipi dei campi. Modello interno: ID normalizzato e prezzo numerico. Concentra la conversione al confine, invece di ripeterla ovunque.

10. `calcolaAttesa(1, 0) = 60000`; `calcolaAttesa(6, 0) = 1800000`; `calcolaAttesa(1, 3600000) = 3600000`. Sono millisecondi.

11. Intervallo: possibilità di sovrapposizione, da gestire. Ciclo con `await`: una operazione alla volta, periodo comprensivo di lavoro e pausa. La scelta dipende dal comportamento voluto.

12. Il test alla soglia distingue `<=` e `<`. È una verifica del requisito, non una ripetizione dell'implementazione.

13. Con 429 non esegui `notified.add`; entri nel catch, incrementi failures, aggiorni `nextCheckAt` e nel finally liberi running.

15. Mini-progetto finale e metodo di lavoro

Ricostruisci un monitor senza guardare bot.js: sorgente HTTP locale, filtro puro, formatter, invio simulato, Set e blocco. Deve funzionare senza servizi esterni. Collega poi un servizio reale alla volta.

Criteri: una notifica per ID nella stessa esecuzione; prezzo alla soglia accettato; valori non validi esclusi; invio fallito non registrato; blocco liberato dopo errore; controlli simultanei non sovrapposti; errori HTTP distinguibili. Dimostra i risultati con test.

Sfida avanzata: persistenza degli ID. Progetta file assente, corrotto o non scrivibile, poi scegli formato e scrittura atomica. Non aggiungere writeFile senza ragionare su errori e riavvii.

Ho usato conoscenza di JavaScript e protocolli, librerie esistenti e generazione assistita del codice. La velocità di scrittura non equivale a certezza: il primo endpoint era sbagliato. Abbiamo osservato una richiesta, corretto il confine HTTP e verificato sul tuo PC. Diagnosi e test fanno parte della programmazione.

Riferimenti: <https://developer.mozilla.org/en-US/docs/Web/JavaScript> • <https://nodejs.org/en/learn> • <https://nodejs.org/api/test.html> • <https://nodecron.com> • <https://core.telegram.org/bots/api>

Repository: <https://github.com/ValePrugna/vinted>

Appendice • 01-filtro.mjs

File completo in docs/laboratorio/01-filtro.mjs. I numeri di riga sono solo per la lettura: non copiarli nel codice. Le righe lunghe proseguono sulla riga visiva successiva.

```
1 import assert from 'node:assert/strict';
2 import { pathToFileURL } from 'node:url';
3
4 // Funzione pura: non legge rete o stato globale e non modifica lista.
5 export function trovaOccasioni(lista, massimo) {
6   return lista.filter(articolo => Number.isFinite(articolo.prezzo)
7     && articolo.prezzo >= 0 && articolo.prezzo <= massimo);
8 }
9
10 if (process.argv[1] && import.meta.url === pathToFileURL(process.argv[1]).href) {
11   const articoli = [
12     { id: 1, titolo: 'Bracciale', prezzo: 20 },
13     { id: 2, titolo: 'Collana', prezzo: 45 },
14     { id: 3, titolo: 'Anello', prezzo: 25 },
15   ];
16   const risultato = trovaOccasioni(articoli, 25);
17   assert.deepEqual(risultato.map(a => a.id), [1, 3]);
18   console.log(risultato);
19 }
```

Appendice • 02-http.mjs

File completo in docs/laboratorio/02-http.mjs. I numeri di riga sono solo per la lettura: non copiarli nel codice. Le righe lunghe proseguono sulla riga visiva successiva.

```
1 import { createServer } from 'node:http';
2 import { once } from 'node:events';
3 import assert from 'node:assert/strict';
4 import { trovaOccasioni } from './01-filtro.mjs';
5
6 // Server di laboratorio: nessuna chiamata a Vinted o Telegram.
7 const server = createServer((request, response) => {
8   const url = new URL(request.url, 'http://127.0.0.1');
9   response.setHeader('Content-Type', 'application/json');
10  if (url.pathname !== '/items') {
11    response.writeHead(404);
12    response.end(JSON.stringify({ error: 'Risorsa assente' }));
13    return;
14  }
15  response.end(JSON.stringify({ items: [
16    { id: 1, titolo: 'Bracciale', prezzo: 20 },
17    { id: 2, titolo: 'Collana', prezzo: 45 },
18  ] }));
19 });
20
21 async function caricaArticoli(url) {
22   const response = await fetch(url, { signal: AbortSignal.timeout(5000) });
23   if (!response.ok) throw new Error(`HTTP ${response.status}`);
24   const data = await response.json();
25   if (!Array.isArray(data.items)) throw new Error('items deve essere un array');
26   return data.items;
27 }
28
29 // Porta 0: il sistema sceglie una porta libera. Bind soltanto su loopback.
30 server.listen(0, '127.0.0.1');
31 await once(server, 'listening');
32 try {
33   const base = `http://127.0.0.1:${server.address().port}`;
34   const url = new URL('/items', base);
35   url.search = new URLSearchParams({ search_text: 'bracciale argento', price_to: '25'
36 }).toString();
37   const articoli = await caricaArticoli(url);
38   const occasioni = trovaOccasioni(articoli, 25);
39   assert.deepEqual(occasioni.map(a => a.id), [1]);
40   console.log('Richiesta HTTP riuscita:', occasioni);
41   await assert.rejects(caricaArticoli(new URL('/missing', base)), /HTTP 404/);
42   console.log('Gestione HTTP 404 verificata');
43 } finally {
44   const closed = once(server, 'close');
45   server.close();
46   server.closeAllConnections();
47   await closed;
48 }
```

Appendice • 03-monitor.mjs

File completo in docs/laboratorio/03-monitor.mjs. I numeri di riga sono solo per la lettura: non copiarli nel codice. Le righe lunghe proseguono sulla riga visiva successiva.

```
1 import { trovaOccasioni } from './01-filtro.mjs';
2
3 export function creaMonitor({ carica, invia, massimo }) {
4   if (typeof carica !== 'function' || typeof invia !== 'function') {
5     throw new TypeError('carica e invia devono essere funzioni');
6   }
7   if (!Number.isFinite(massimo) || massimo < 0) throw new TypeError('Soglia non valida');
8   const notificati = new Set();
9   let attivo = false;
10
11   async function controlla() {
12     if (attivo) return { saltato: true, inviati: 0 };
13     attivo = true;
14     let conteggio = 0;
15     try {
16       const articoli = await carica();
17       if (!Array.isArray(articoli)) throw new TypeError('Lista articoli non valida');
18       for (const articolo of trovaOccasioni(articoli, massimo)) {
19         if (articolo.id === null || articolo.id === undefined) continue;
20         const id = String(articolo.id);
21         if (!/^\\d+$/ .test(id) || notificati.has(id)) continue;
22         await invia(articolo);
23         notificati.add(id);
24         conteggio++;
25       }
26       return { saltato: false, inviati: conteggio };
27     } finally {
28       attivo = false;
29     }
30   }
31   return { controlla, notificati };
32 }
```

Appendice • 03-demo.mjs

File completo in docs/laboratorio/03-demo.mjs. I numeri di riga sono solo per la lettura: non copiarli nel codice. Le righe lunghe proseguono sulla riga visiva successiva.

```
1 import assert from 'node:assert/strict';
2 import { creaMonitor } from './03-monitor.mjs';
3
4 const monitor = creaMonitor({
5   massimo: 25,
6   carica: async () => [
7     { id: 1, titolo: 'Bracciale', prezzo: 20 },
8     { id: 2, titolo: 'Collana', prezzo: 45 },
9   ],
10  invia: async articolo => console.log(`NOTIFICA SIMULATA: ${articolo.titolo}`),
11 });
12 const primo = await monitor.controlla();
13 const secondo = await monitor.controlla();
14 assert.equal(primo.inviati, 1);
15 assert.equal(secondo.inviati, 0);
16 console.log('Primo controllo:', primo);
17 console.log('Secondo controllo:', secondo);
```

Appendice • verifica.mjs

File completo in docs/laboratorio/verifica.mjs. I numeri di riga sono solo per la lettura: non copiarli nel codice. Le righe lunghe proseguono sulla riga visiva successiva.

```
1 import test from 'node:test';
2 import assert from 'node:assert/strict';
3 import { trovaOccasioni } from './01-filtro.mjs';
4 import { creaMonitor } from './03-monitor.mjs';
5
6 test('Filtro: include soglia e zero, esclude prezzi non validi', () => {
7   const articoli = [0, 25, 25.01, -1, NaN, '20'].map((prezzo, id) => ({ id, prezzo }));
8   assert.deepEqual(trovaOccasioni(articoli, 25).map(a => a.id), [0, 1]);
9 });
10
11 test('Due controlli notificano una sola volta e normalizzano ID', async () => {
12   const inviati = [];
13   let chiamate = 0;
14   const monitor = creaMonitor({
15     massimo: 25,
16     carica: async () => [{ id: ++chiamate === 1 ? 42 : '42', prezzo: 20 }],
17     invia: async articolo => { inviati.push(articolo.id); },
18   });
19   await monitor.controlla();
20   await monitor.controlla();
21   assert.deepEqual(inviati, [42]);
22   assert.deepEqual([...monitor.notificati], ['42']);
23 });
24
25 test('Invio fallito: ID non registrato e controllo successivo possibile', async () => {
26   let tentativi = 0;
27   const monitor = creaMonitor({
28     massimo: 25,
29     carica: async () => [{ id: 1, prezzo: 20 }],
30     invia: async () => { if (++tentativi === 1) throw new Error('Invio fallito'); },
31   });
32   await assert.rejects(monitor.controlla(), /Invio fallito/);
33   assert.equal(monitor.notificati.size, 0);
34   assert.equal((await monitor.controlla()).inviati, 1);
35   assert.ok(monitor.notificati.has('1'));
36 });
37
38 test('Acquisizione fallita: il blocco viene liberato', async () => {
39   let tentativi = 0;
40   const monitor = creaMonitor({
41     massimo: 25,
42     carica: async () => { if (++tentativi === 1) throw new Error('Rete'); return []; },
43     invia: async () => {},
44   });
45   await assert.rejects(monitor.controlla(), /Rete/);
46   assert.equal((await monitor.controlla()).saltato, false);
47 });
48
49 test('Un controllo attivo impedisce sovrapposizioni', async () => {
50   let sblocca;
51   const attesa = new Promise(resolve => { sblocca = resolve; });
52   let richieste = 0;
53   const monitor = creaMonitor({
54     massimo: 25,
55     carica: async () => { richieste++; await attesa; return []; },
56     invia: async () => {},
57   });
58   const primo = monitor.controlla();
59   assert.equal((await monitor.controlla()).saltato, true);
60   sblocca();
61   await primo;
62   assert.equal(richieste, 1);
63 });
```